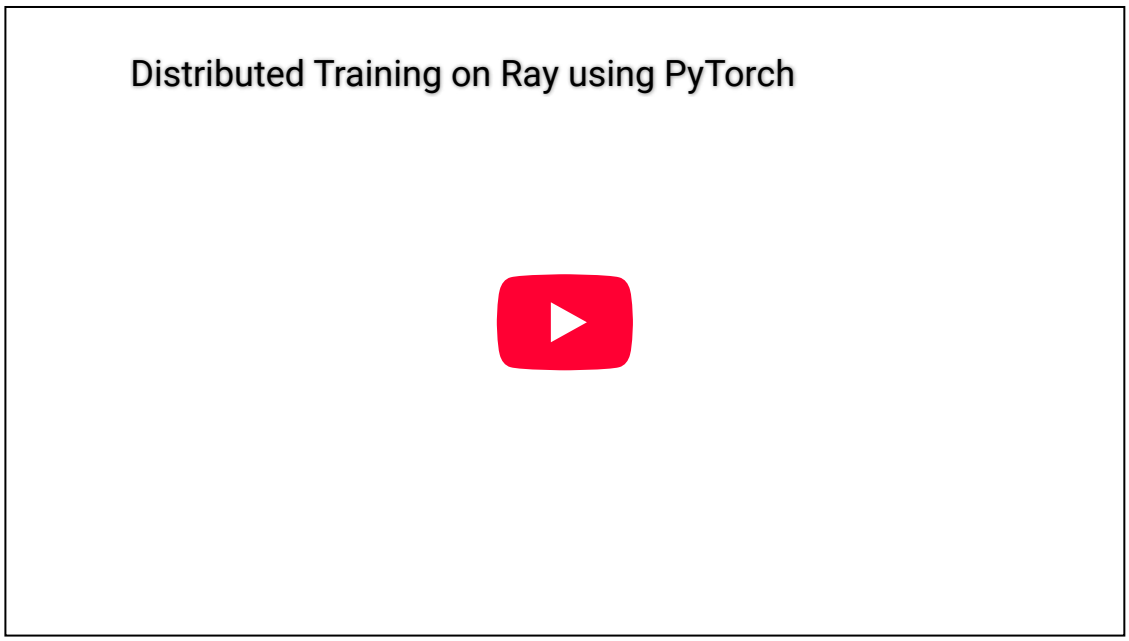
PyTorch

In this Get Started guide, you will perform Distributed Training using PyTorch against your remote Ray endpoint. This guide assumes the following:

- You have already created a "Ray as Service" tenant using Rafay
- You have the https URL and Access Credentials to the remote endpoint.
- You have Python 3 installed on your laptop

Watch a brief video showcasing what you will experience with this guide.



Review Code

Download the source code file "pytorch_test.py" and open it in your favorite IDE such as VS Code to review it. This code demonstrates how to use **Ray** for distributed training of a simple **PyTorch model**.

It sets up parallel training tasks where each worker trains a separate instance of the model and then aggregates the trained parameters from all workers. This approach is useful for speeding up training by distributing the computation across multiple processes or machines. Here is a brief description of what this code can help you accomplish.

Distributed Model Training

Trains four independent instances of a simple PyTorch model using Ray's distributed capabilities, running each model training on a separate Ray worker. Parallel training means that if each worker takes 5 seconds to train, the overall time will be close to 5 seconds rather than 20 seconds.

Model Parameter Aggregation

The trained models' parameters are averaged to produce a single set of parameters, combining the results of all the workers. This is akin to methods used in **federated learning** or **parameter server models**.

Scalability

This example can scale across multiple machines or GPUs. Since we are using a Ray cluster, you can perform larger-scale machine learning training tasks.

Benchmarking

By measuring the time taken for each worker and aggregating the results, users can analyze the performance and efficiency of the distributed training setup.

Step-by-Step

Below is a detailed explanation of the example.

Import Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import ray
import numpy as np
```

`torch` : The core library for building and training neural networks in PyTorch.

`ray` : A distributed computing library that allows us to run tasks in parallel.

`numpy` : Used for generating random data (though here PyTorch handles data generation).

Initialize Ray

```
ray.init()
```

Initializes the Ray runtime, enabling the script to submit tasks to the Ray cluster.

Define a Simple PyTorch Model

```
class SimpleModel(nn.Module):
    def __init__(self):
        super(SimpleModel, self).__init__()
        self.fc1 = nn.Linear(10, 50)
        self.fc2 = nn.Linear(50, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

`SimpleModel` is a simple feedforward neural network with:

- `fc1` : A linear layer with 10 input features and 50 output features.
- `fc2` : A linear layer with 50 input features and 1 output feature.

`forward` defines the forward pass of the model using a ReLU activation between layers.

Define a Remote Training Function

```
@ray.remote
def train_worker(rank, num_epochs=5):
    ...
    return model.state_dict()
```

`@ray.remote` : Decorates the `train_worker` function, allowing it to run on a separate Ray worker.

Function Parameters: - `rank` : Identifies each worker (useful for logging). - `num_epochs` : Specifies the number of epochs for training.

Training Data: Generates **100 samples** with **10 features** each for `X` and corresponding target values `y` .

- **DataLoader**: Wraps the data in a `DataLoader` for batching and shuffling, with a **batch size of 16**.

- **Model, Loss, Optimizer**: Initializes a `SimpleModel`, **MSE Loss** (mean squared error), and **SGD optimizer**.

Training Loop: - Iterates over the data for `num_epochs`. - For each batch, computes the model's predictions, calculates the loss, backpropagates the error, and updates the model parameters. - Tracks and prints the **average loss per epoch**.

Returns the Trained Model Parameters: Uses `model.state_dict()` to return the trained parameters (weights and biases) of the model.

Aggregate Model Parameters

```
def average_model_params(models):
    avg_state_dict = models[0]
    for key in avg_state_dict.keys():
        for i in range(1, len(models)):
            avg_state_dict[key] += models[i][key]
        avg_state_dict[key] = avg_state_dict[key] / len(models)
    return avg_state_dict
```

Averages the Model Parameters across all the workers to create a consensus model. Iterates over each parameter (e.g., weights) and sums the corresponding parameter values from all workers Divides the summed values by the number of models (workers) to compute the average.

Note that this approach is useful in **distributed training** or **federated learning**, where each worker trains on a subset of data, and their models are combined to form a global model.

Main Execution Block

```
if __name__ == "__main__":
    # Number of workers
    num_workers = 4

    # Launch distributed training on multiple workers
    futures = [train_worker.remote(rank=i, num_epochs=5) for i in
range(num_workers)]

    # Collect the trained model parameters from each worker
    results = ray.get(futures)

    # Aggregate the model parameters from all workers (averaging)
    avg_model_params = average_model_params(results)

    print("Aggregated model parameters from all workers!")
```

```
    # Shutdown Ray
    ray.shutdown()
```

Number of Workers: Defines the number of parallel workers as **4**.

Submit Remote Training Tasks: Creates a list of **futures** (task references) by calling `train_worker.remote()` for each worker with different `rank` values. Each worker trains a model independently.

Retrieve Results: - Uses `ray.get(futures)` to wait for the completion of all tasks and retrieve their results. - `results` is a list of model parameter dictionaries (one from each worker).

Aggregate Parameters: Calls `average_model_params` to average the parameters across all trained models. - **Output**: Prints a message indicating that the aggregation is complete. - **Shutdown Ray**: Shuts down Ray, releasing any resources used during the training.

Job Submission Code

Download the source code file "run.py" and open it in your favorite IDE such as VS Code to review it. As you can see from the code snippet below, we will be using Ray's **Job Submission Client** to submit a job to the remote Ray endpoint.

```
from ray.job_submission import JobSubmissionClient
import ray
import urllib3

# Suppress the warning about unverified HTTPS requests
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

# Ray client
client = JobSubmissionClient(
    "https://<URL> for Ray Endpoint>",
    headers={"Authorization": "Basic <Base64 Encoded Credentials>"},
    verify=False  # Disable SSL verification
)

# Submit job
client.submit_job(entrypoint="python pytorch_test.py", runtime_env=
{"working_dir": "./"})
```

Now, update the **authorization credentials** with the base64 encoded credentials for your Ray endpoint. You can use the following command to perform the encoding.

```
echo -n 'admin:PASSWORD' | base64
```

Submit Job

In order to submit the job to your remote Ray endpoint,

- First, in your web browser, access the Ray Dashboard's URL and keep it open. We will monitor the status and progress of the submitted job here.
- Now, open Terminal and enter the following command

```
python3 ./run.py
```

This will submit the job to the configured Ray endpoint and you can review progress and the results on the Ray Dashboard. Once the Ray endpoint receives the job, it will be pending for a few seconds.

Shown below is an example of the expected output from a typical run. For each worker, the code will print the average loss for each epoch:

```
Worker 0 - Epoch 1/5, Loss: 0.4321
Worker 1 - Epoch 1/5, Loss: 0.5678
...
Worker 3 - Epoch 5/5, Loss: 0.1234
```

Important

These values above represent the loss as the model learns to fit the random data generated for training.

After all workers complete their training, the following message is printed:

```
Aggregated model parameters from all workers!
```